

## 24D.1 — crear las transformaciones

DROP TABLE IF EXISTS

bu\_frd.model\_behavior\_features\_transformed\_2022\_2025;

CREATE TABLE

bu\_frd.model\_behavior\_features\_transformed\_2022\_2025

USING PARQUET

AS

SELECT

\*,

/\* =====  
ACTIVE ACCOUNTS  
Magnitude of change  
===== \*/

ABS(  
active\_accounts\_change\_vs\_baseline  
) AS abs\_active\_accounts\_change,

/\* =====  
BALANCE  
Magnitude, log compressed  
===== \*/

CASE  
WHEN balance\_change\_vs\_baseline IS NULL  
THEN NULL

ELSE LN(  
1.0 + ABS(balance\_change\_vs\_baseline)  
)  
END AS log1p\_abs\_balance\_change

FROM

bu\_frd.model\_behavior\_features\_selected\_2022\_2025;

QA estructural:

SELECT

COUNT(\*) AS rows,  
COUNT(DISTINCT party\_id) AS parties,

SUM(  
,

```

CASE
  WHEN abs_active_accounts_change IS NULL
  THEN 1 ELSE 0
END
) AS abs_active_missing,

SUM(
  CASE
    WHEN log1p_abs_balance_change IS NULL
    THEN 1 ELSE 0
  END
) AS log_balance_missing

FROM
  bu_frd.model_behavior_features_transformed_2022_2025;

```

Esperamos:

```

rows          688,589
parties        688,589
abs_active_missing    0
log_balance_missing  64,915 aproximadamente en TRAIN
                    + correspondientes VALIDATION/TEST

```

24D.2 — deciles de las dos transformaciones en TRAIN

WITH long\_features AS (

```

SELECT
  party_id,
  label_3m,

  STACK(
    2,

    'abs_active_accounts_change',
    CAST(abs_active_accounts_change AS DOUBLE),

    'log1p_abs_balance_change',
    CAST(log1p_abs_balance_change AS DOUBLE)

  ) AS (feature_name, feature_value)

```

```

FROM
  bu_frd.model_behavior_features_transformed_2022_2025

```

```
WHERE model_split = 'TRAIN'
),

available AS (

SELECT *
FROM long_features

WHERE feature_value IS NOT NULL
),
```

```
ranked AS (

SELECT
    *,

    NTILE(10) OVER (
        PARTITION BY feature_name
        ORDER BY feature_value
    ) AS decile

FROM available
),
```

```
baseline AS (

SELECT
    feature_name,

    AVG(
        CAST(label_3m AS DOUBLE)
    ) AS available_positive_rate

FROM available

GROUP BY feature_name
)
```

```
SELECT
    r.feature_name,
    r.decile,

    COUNT(*) AS parties,
    SUM(r.label_3m) AS cases,

    MIN(r.feature_value) AS min_value,
```

```

percentile_approx(
    r.feature_value,
    0.50
) AS median_value,

MAX(r.feature_value) AS max_value,

ROUND(
    100.0 *
    AVG(CAST(r.label_3m AS DOUBLE)),
    4
) AS positive_rate_pct,

ROUND(
    AVG(CAST(r.label_3m AS DOUBLE))
    /
    MAX(b.available_positive_rate),
    4
) AS lift_vs_available_population

FROM ranked r

INNER JOIN baseline b
    ON r.feature_name = b.feature_name

GROUP BY
    r.feature_name,
    r.decile

ORDER BY
    r.feature_name,
    r.decile;

```

Se agrega

WITH val AS (

SELECT

label\_3m,

```
/* =====  
ABS ACTIVE CHANGE  
Boundaries frozen from TRAIN  
===== */
```

CASE

WHEN abs\_active\_accounts\_change = 0  
THEN 'A0\_ZERO'

WHEN abs\_active\_accounts\_change <= 0.2222222  
THEN 'A1\_GT0\_TO\_0.222'

WHEN abs\_active\_accounts\_change <= 0.4444444  
THEN 'A2\_0.222\_TO\_0.444'

WHEN abs\_active\_accounts\_change <= 0.8888889  
THEN 'A3\_0.444\_TO\_0.889'

WHEN abs\_active\_accounts\_change <= 1.3333333  
THEN 'A4\_0.889\_TO\_1.333'

ELSE 'A5\_GT\_1.333'

END AS active\_group,

```
/* =====  
LOG ABS BALANCE  
TRAIN decile boundaries  
===== */
```

CASE

WHEN log1p\_abs\_balance\_change IS NULL  
THEN 'MISSING'

WHEN log1p\_abs\_balance\_change <= 3.7746748  
THEN 'B01'

WHEN log1p\_abs\_balance\_change <= 4.9714966  
THEN 'B02'

```

        WHEN log1p_abs_balance_change <= 5.6922346
            THEN 'B03'

        WHEN log1p_abs_balance_change <= 6.2697274
            THEN 'B04'

        WHEN log1p_abs_balance_change <= 6.7816880
            THEN 'B05'

        WHEN log1p_abs_balance_change <= 7.2708540
            THEN 'B06'

        WHEN log1p_abs_balance_change <= 7.7770709
            THEN 'B07'

        WHEN log1p_abs_balance_change <= 8.3935832
            THEN 'B08'

        WHEN log1p_abs_balance_change <= 9.2550650
            THEN 'B09'

        ELSE 'B10'
    END AS balance_group

FROM
    bu_frd.model_behavior_features_transformed_2022_2025

WHERE model_split = 'VALIDATION'
),

long_data AS (

    SELECT
        label_3m,

        STACK(
            2,

            'abs_active_accounts_change',
            active_group,

            'log1p_abs_balance_change',
            balance_group

        ) AS (feature_name, feature_group)

    FROM val
),

```

baseline AS (

```
SELECT
    feature_name,

    AVG(
        CASE
            WHEN feature_group <> 'MISSING'
            THEN CAST(label_3m AS DOUBLE)
        END
    ) AS available_positive_rate
```

FROM long\_data

GROUP BY feature\_name

)

```
SELECT
    x.feature_name,
    x.feature_group,

    COUNT(*) AS parties,
    SUM(x.label_3m) AS cases,
```

```
ROUND(
    100.0 *
    AVG(CAST(x.label_3m AS DOUBLE)),
    4
) AS positive_rate_pct,
```

```
ROUND(
    AVG(CAST(x.label_3m AS DOUBLE))
    /
    MAX(b.available_positive_rate),
    4
) AS lift_vs_available_population
```

FROM long\_data x

```
INNER JOIN baseline b
    ON x.feature_name = b.feature_name
```

```
GROUP BY
    x.feature_name,
    x.feature_group
```

ORDER BY

```
x.feature_name,  
x.feature_group;
```